Backtracking

Nawal Dandekar

Backtracking

- Backtracking algorithms are like problem-solving strategies that help explore different options to find the best solution.
- They work by trying out different paths and if one doesn't work, they backtrack and try another until they find the right one.
- It's like solving a puzzle by testing different pieces until they fit together perfectly.

What is Backtracking Algorithm?

- Backtracking is a problem-solving algorithmic technique that involves finding a solution incrementally by trying different options and undoing them if they lead to a dead end.

- It is commonly used in situations where you need to explore multiple possibilities to solve a problem, like searching for a path in a maze or solving puzzles like Sudoku. When a dead end is reached, the algorithm backtracks to the previous decision point and explores a different path until a solution is found or all possibilities have been exhausted.

How Does a Backtracking Algorithm Work?

- A backtracking algorithm works by recursively exploring all possible solutions to a problem.
- It starts by choosing an initial solution, and then it explores all possible extensions of that solution.
- If an extension leads to a solution, the algorithm returns that solution. If an extension does not lead to a solution, the algorithm backtracks to the previous solution and tries a different extension.

How Does a Backtracking Algorithm Work?

- The following is a general outline of how a backtracking algorithm works:

1. Choose an initial solution.
2. Explore all possible extensions of the current solution.
3. If an extension leads to a solution, return that solution.
4. If an extension does not lead to a solution, backtrack to the previous solution and try a different extension.
5. Repeat steps 2-4 until all possible solutions have been explored.

When to Use a Backtracking Algorithm?

- Backtracking algorithms are best used to solve problems that have the following characteristics:

- There are multiple possible solutions to the problem.
- The problem can be broken down into smaller subproblems.
- The subproblems can be solved independently.

Applications of Backtracking Algorithm

- Backtracking algorithms are used in a wide variety of applications, including:
- Solving puzzles (e.g., Sudoku, crossword puzzles)
- Finding the shortest path through a maze
- Scheduling problems
- Resource allocation problems
- Network optimization problems
- Combinatorial problems, such as generating permutations, combinations, or subsets.

N Queen Problem

- Given an integer **n**, place n queens on an **n × n chessboard** such that no two queens attack each other. A queen can attack another queen if they are placed in the same **row**, the same **column**, or on the same **diagonal**.
- Find all possible distinct arrangements of the queens on the board that satisfy these conditions.
- The output should be an array of solutions, where each solution is represented as an array of integers of size n, and the i-th integer denotes the column position of the queen in the i-th row. If no solution exists, return an empty array.

Subset Sum Problem

- In the sum of subsets problem, there is a given set with some non-negative integer elements. And another sum value is also provided, our task is to find all possible subsets of the given set whose sum is the same as the given sum value.

Input Output Scenario

- Suppose the given set and sum value is −
- Set = {1, 9, 7, 5, 18, 12, 20, 15}
- sum value = 35

- All possible subsets of the given set, where sum of each element for every subset is the same as the given sum value are given below −
- {1 9 7 18}
- {1 9 5 20}
- {5 18 12}

Backtracking Approach to solve Subset Sum Problem

- In the naive method to solve a subset sum problem, the algorithm generates all the possible permutations and then checks for a valid solution one by one. Whenever a solution satisfies the constraints, mark it as a part of the solution.
- In solving the subset sum problem, the backtracking approach is used for selecting a valid subset. When an item is not valid, we will backtrack to get the previous subset and add another element to get the solution.
- In the worst-case scenario, backtracking approach may generate all combinations, however, in general, it performs better than the naive approach.

Steps to solve

- First, take an empty subset.
- Include the next element, which is at index 0 to the empty set.
- If the subset is equal to the sum value, mark it as a part of the solution.
- If the subset is not a solution and it is less than the sum value, add next element to the subset until a valid solution is found.
- Now, move to the next element in the set and check for another solution until all combinations have been tried.

Graph Coloring Problem

- Graph coloring refers to the problem of coloring vertices of a graph in such a way that no two adjacent vertices have the same color. This is also called the vertex coloring problem. If coloring is done using at most m colors, it is called m-coloring.

Algorithm of Graph Coloring using Backtracking:

- Assign colors one by one to different vertices, starting from vertex 0. Before assigning a color, check if the adjacent vertices have the same color or not. If there is any color assignment that does not violate the conditions, mark the color assignment as part of the solution. If no assignment of color is possible then backtrack and return false.